

OSSP Skill Week 12 - Semaphores and Producer-Consumer

Task-1

Aim

To study semaphores and their operations, and to use a counting semaphore to manage a critical section in a multithreaded program.

Program

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #include <fcntl.h>
6  #include <unistd.h>
7
8  #define NUM_THREADS 4
9  #define MAX_ACCESS 2          /* Threads allowed in at once */
10 #define SEM_NAME    "/ossp_counting_sem"
11
12 static sem_t *semaphore;
13
14 static void *access_resource(void *arg)
15 {
16     int id = *(int *)arg;
17     printf("Thread %d is waiting for the resource.\n", id);
18     if (sem_wait(semaphore) != 0) {          /* wait / P operation */
19         perror("sem_wait");
20         return NULL;
21     }
22     printf("Thread %d entered the critical section.\n", id);
23     sleep(1);                               /* Use the shared resource */
24     printf("Thread %d leaving the critical section.\n", id);
25     if (sem_post(semaphore) != 0)           /* signal / V operation */
26         perror("sem_post");
27     return NULL;
28 }
29
30 int main(void)
31 {
32     pthread_t threads[NUM_THREADS];
33     int ids[NUM_THREADS];
34     int created = 0;
35
36     /* Remove a semaphore left over from an earlier run */
37     sem_unlink(SEM_NAME);
38
39     semaphore = sem_open(SEM_NAME, O_CREAT | O_EXCL, 0644, MAX_ACCESS);
40     if (semaphore == SEM_FAILED) {
41         perror("sem_open");
42         return 1;
43     }
44     printf("Counting semaphore initialized with value %d.\n", MAX_ACCESS);
45
46     for (int i = 0; i < NUM_THREADS; i++) {
47         ids[i] = i + 1;
48         if (pthread_create(&threads[i], NULL, access_resource, &ids[i]) != 0) {
49             fprintf(stderr, "Failed to create thread %d\n", i + 1);
50             break;
51         }
52         created++;
53     }
54
55     for (int i = 0; i < created; i++)
56         pthread_join(threads[i], NULL);
57
58     sem_close(semaphore);
59     sem_unlink(SEM_NAME);
60
61     printf("\nAll threads completed successfully.\n");
62     return 0;
63 }
```

Output

```
===== Testing Completed: 2/2 passed =====  
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc semaphore_demo.c -o semaphore_demo  
● hrushika@Hrushikas-MacBook-Air pjsdj % ./semaphore_demo  
Counting semaphore initialized with value 2.  
  
Thread 1 is waiting for the resource.  
Thread 1 entered the critical section.  
Thread 2 is waiting for the resource.  
Thread 2 entered the critical section.  
Thread 3 is waiting for the resource.  
Thread 4 is waiting for the resource.  
Thread 1 leaving the critical section.  
Thread 2 leaving the critical section.  
Thread 3 entered the critical section.  
Thread 4 entered the critical section.  
Thread 3 leaving the critical section.  
Thread 4 leaving the critical section.  
  
All threads completed successfully.  
❖ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

All four threads requested the resource, but only two entered the critical section at first, because the semaphore started at 2. The remaining two threads entered only after the first two left and signalled the semaphore. The thread order changes from run to run because the operating system schedules the threads.

Result

Semaphore synchronization was implemented successfully with a counting semaphore. The semaphore controlled access to the critical section and allowed at most two threads to use the shared resource at the same time.

Task-2

Aim

To implement the Producer-Consumer problem using counting semaphores and demonstrate synchronization between producer and consumer threads.

Program

```
producer_consumer_semaphore.c 1 main(void)
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 10
#define EMPTY_NAME "/ossp_pc_empty"
#define FULL_NAME "/ossp_pc_full"

static int buffer[BUFFER_SIZE];
static int in = 0; /* Next slot to fill */
static int out = 0; /* Next slot to empty */

static sem_t *empty; /* Free slots */
static sem_t *full; /* Filled slots */
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

static void *producer(void *arg)
{
    (void)arg;

    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(empty); /* Wait for a free slot */
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        printf("Produced: %d\n", item);

        pthread_mutex_unlock(&mutex);
        sem_post(full); /* One more item available */

        usleep(100000); /* 0.1 s */
    }

    return NULL;
}

static void *consumer(void *arg)
{
    (void)arg;

    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(full); /* Wait for an item */
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        printf("Consumed: %d\n", item);

        pthread_mutex_unlock(&mutex);
        sem_post(empty); /* One more free slot */

        usleep(150000); /* 0.15 s */
    }

    return NULL;
}

static void cleanup(void)
{
    if (empty != SEM_FAILED && empty != NULL)
        sem_close(empty);
    if (full != SEM_FAILED && full != NULL)
        sem_close(full);
    sem_unlink(EMPTY_NAME);
    sem_unlink(FULL_NAME);
}

int main(void)
{
    pthread_t producer_thread, consumer_thread;

    /* Remove semaphores left over from an earlier run */
    sem_unlink(EMPTY_NAME);
    sem_unlink(FULL_NAME);
```

```

}

int main(void)
{
    pthread_t producer_thread, consumer_thread;

    /* Remove semaphores left over from an earlier run */
    sem_unlink(EMPTY_NAME);
    sem_unlink(FULL_NAME);

    empty = sem_open(EMPTY_NAME, O_CREAT | O_EXCL, 0644, BUFFER_SIZE);
    full = sem_open(FULL_NAME, O_CREAT | O_EXCL, 0644, 0);
    if (empty == SEM_FAILED || full == SEM_FAILED) {
        perror("sem_open");
        cleanup();
        return 1;
    }

    printf("Starting Producer-Consumer using semaphores...\n\n");

    if (pthread_create(&producer_thread, NULL, producer, NULL) != 0 ||
        pthread_create(&consumer_thread, NULL, consumer, NULL) != 0) {
        fprintf(stderr, "Failed to create threads.\n");
        cleanup();
        return 1;
    }

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    cleanup();
    pthread_mutex_destroy(&mutex);

    printf("\nProducer and Consumer completed successfully.\n");
    return 0;
}
```

Output

```
All threads completed successfully.
hrushika@Hrushikas-MacBook-Air pjsdj % gcc producer_consumer_sem.c -o producer_consumer_sem
hrushika@Hrushikas-MacBook-Air pjsdj % ./producer_consumer_sem
Starting Producer-Consumer using semaphores...

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Produced: 5
Consumed: 4
Produced: 6
Consumed: 5
Produced: 7
Produced: 8
Consumed: 6
Produced: 9
Consumed: 7
Produced: 10
Consumed: 8
Consumed: 9
Consumed: 10

Producer and Consumer completed successfully.
hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The producer created items 1 to 10 and the consumer removed them in the same order. Because the producer is faster (0.1 s) than the consumer (0.15 s), the producer gradually moved ahead, but it never placed more than 5 items in the buffer. The consumer never removed an item before it was produced.

Result

The Producer-Consumer problem was implemented successfully using counting semaphores and a mutex. The semaphores synchronized the producer and consumer threads, preventing buffer overflow, buffer underflow and race conditions.